JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Interface Spliterator.OfDouble

All Superinterfaces:

Spliterator<Double>, Spliterator.OfPrimitive<Double,DoubleConsumer,Spliterator.OfDouble>

All Known Implementing Classes:

Spliterators.AbstractDoubleSpliterator

Enclosing interface:

Spliterator<T>

```
public static interface Spliterator.OfDouble
extends Spliterator.OfPrimitive<Double,DoubleConsumer,Spliterator.OfDouble>
```

A Spliterator specialized for `double` values.

Since:

1.8

Nested Class Summary

Nested classes/interfaces inherited from interface java.util.Spliterator

Spliterator.OfDouble, Spliterator.OfInt, Spliterator.OfLong, Spliterator.OfPrimitive<T,T_CONS,T_SPLITR extends Spliterator.OfPrimitive<T,T_CONS,T_SPLITR>>

Field Summary

Fields inherited from interface java.util.Spliterator

CONCURRENT, DISTINCT, IMMUTABLE, NONNULL, ORDERED, SIZED, SORTED, SUBSIZED

Method Summary

Default Methods

Modifier and Type	Method	Description
default void	forEachRemaining(Consumer<? super Double> action)	Performs the given action for each remaining element, sequentially in the current thread, until all elements have been processed or the action throws an exception.
default void	forEachRemaining(DoubleConsumer action)	Performs the given action for each remaining element, sequentially in the current thread, until all elements have been processed or the action

		throws an exception.
default boolean	**tryAdvance**(**Consumer**<? super **Double**> action)	If a remaining element exists, performs the given action on it, returning `true`; else returns `false`.
boolean	**tryAdvance**(**DoubleConsumer** action)	If a remaining element exists, performs the given action on it, returning `true`; else returns `false`.
Spliterator.OfDouble	**trySplit**()	If this spliterator can be partitioned, returns a Spliterator covering elements, that will, upon return from this method, not be covered by this Spliterator.

Methods inherited from interface java.util.Spliterator

`characteristics`, `estimateSize`, `getComparator`, `getExactSizeIfKnown`, `hasCharacteristics`

Method Detail

trySplit

`Spliterator.OfDouble trySplit()`

Description copied from interface: `Spliterator`

If this spliterator can be partitioned, returns a Spliterator covering elements, that will, upon return from this method, not be covered by this Spliterator.

If this Spliterator is `Spliterator.ORDERED`, the returned Spliterator must cover a strict prefix of the elements.

Unless this Spliterator covers an infinite number of elements, repeated calls to `trySplit()` must eventually return `null`. Upon non-null return:

- the value reported for `estimateSize()` before splitting, must, after splitting, be greater than or equal to `estimateSize()` for this and the returned Spliterator; and
- if this Spliterator is `SUBSIZED`, then `estimateSize()` for this spliterator before splitting must be equal to the sum of `estimateSize()` for this and the returned Spliterator after splitting.

This method may return `null` for any reason, including emptiness, inability to split after traversal has commenced, data structure constraints, and efficiency considerations.

Specified by:

`trySplit in interface Spliterator<Double>`

Specified by:

`trySplit in interface Spliterator.OfPrimitive<Double,DoubleConsumer,Spliterator.OfDouble>`

Returns:

a Spliterator covering some portion of the elements, or null if this spliterator cannot be split

tryAdvance

`boolean tryAdvance(DoubleConsumer action)`

Description copied from interface: `Spliterator.OfPrimitive`

If a remaining element exists, performs the given action on it, returning `true`; else returns `false`. If this Spliterator is `Spliterator.ORDERED` the action is performed on the next element in encounter order. Exceptions thrown by the action are relayed to the caller.

Specified by:

tryAdvance in
interface Spliterator.OfPrimitive<Double,DoubleConsumer,Spliterator.OfDouble>

Parameters:

action - The action

Returns:

false if no remaining elements existed upon entry to this method, else true.

forEachRemaining

default void forEachRemaining(DoubleConsumer action)

Description copied from interface: Spliterator.OfPrimitive

Performs the given action for each remaining element, sequentially in the current thread, until all elements have been processed or the action throws an exception. If this Spliterator is Spliterator.ORDERED, actions are performed in encounter order. Exceptions thrown by the action are relayed to the caller.

Specified by:

forEachRemaining in
interface Spliterator.OfPrimitive<Double,DoubleConsumer,Spliterator.OfDouble>

Parameters:

action - The action

tryAdvance

default boolean tryAdvance(Consumer<? super Double> action)

If a remaining element exists, performs the given action on it, returning true; else returns false. If this Spliterator is Spliterator.ORDERED the action is performed on the next element in encounter order. Exceptions thrown by the action are relayed to the caller.

Specified by:

tryAdvance in interface Spliterator<Double>

Implementation Requirements:

If the action is an instance of DoubleConsumer then it is cast to DoubleConsumer and passed to tryAdvance(java.util.function.DoubleConsumer); otherwise the action is adapted to an instance of DoubleConsumer, by boxing the argument of DoubleConsumer, and then passed to tryAdvance(java.util.function.DoubleConsumer).

Parameters:

action - The action

Returns:

false if no remaining elements existed upon entry to this method, else true.

forEachRemaining

```
default void forEachRemaining(Consumer<? super Double> action)
```

Performs the given action for each remaining element, sequentially in the current thread, until all elements have been processed or the action throws an exception. If this Spliterator is `Spliterator.ORDERED`, actions are performed in encounter order. Exceptions thrown by the action are relayed to the caller.

Specified by:

`forEachRemaining` in interface `Spliterator<Double>`

Implementation Requirements:

If the action is an instance of DoubleConsumer then it is cast to DoubleConsumer and passed to `forEachRemaining(java.util.function.DoubleConsumer)`; otherwise the action is adapted to an instance of DoubleConsumer, by boxing the argument of DoubleConsumer, and then passed to `forEachRemaining(java.util.function.DoubleConsumer)`.

Parameters:

action - The action

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.